



기술 선택 보고서 기본 구조

1. 프로젝트 개요
2. 기술 선택 이유
3. FastAPI의 비동기 아키텍처와 구조적 장점
4. 성능 목표
5. 서버 튜닝 및 성능 실험
6. 전체 실험 요약

1. 프로젝트 개요

- 프로젝트명: 이미지 기반 감정 풍등 음악 생성 서비스
- 핵심 기능
 - 사용자가 이미지를 업로드
 - AI가 음악 생성
 - S3 업로드 후 실시간 알림 (SSE)
- 기술적 과제
 - AI 추론 시간 (15 ~ 25초)으로 인한 응답 지연
 - 사용자 UX를 위한 빠른 응답 및 실시간 상태 알림 필요
 - 10 TPS 이상의 요청 처리 가능해야 함

2. 기술 선택 이유

2-1. Python 기반 선택 이유

항목	선택 이유
AI 서버 표준	MusicGen이 PyTorch 기반으로, AI 모델 추론과 커스터마이징은 Python이 사실상 표준임
통일성	AI 서버와 API 서버를 동일 언어(Python)로 개발해 코드 공유와 배포 관리가 단순해짐
트래픽 특성 적합	API 3개 중 2개는 조회용이고, 1개는 이미지 업로드 후 Celery에 비동기 태스크를 등록하는 용도이다. MusicGen 추론은 비동기로 처리되며, 외부 연동은 S3 업로드뿐이라 CPU-bound 연산이 거의 없다. 현재 TPS가 낮은 단계에서는 Python으로도 충분히 안정적인 처리 성능을 기대할 수 있다.

AI 서버는 Python이 사실상 표준이며, API 서버 또한 I/O-bound 작업 위주로 구성되어 있어 Python 선택이 타당하다고 판단

2-2. Fast API vs Django vs Spring Boot

항목	FastAPI	Django	Spring Boot
비동기 지원	✅ 기본 지원 (async/await 기반)	❖ 기본은 동기 일부 지원 (ASGI 설정 필요)	✅ 지원 (WebFlux 등, 설정 복잡)
SSE 구현	✅ 간편 (StreamingResponse 등)	❖ 가능 (Django Channels 필요)	✅ 가능 (Server-Sent Events 지원)
AI 연동	✅ PyTorch 등 직접 연동 용이	✅ 가능 (Python 기반 모델 연동 용이)	❖ 별도 Python 서버 연동 필요
서버 경량성	✅ 경량 ASGI	❖ 중간 (기본 WSGI, 구조 무거움)	❌ 무거움 (JVM 기반, 빌드/런타임 큼)

우리 서비스는 AI 추론 중심, 이미지 업로드 후 비동기 태스크 처리와 실시간 알림(SSE)이 주요 기능입니다. FastAPI는 Python 생태계 기반이라 AI 모델과 직접 연동 가능하며, ASGI 기반 비동기·논블로킹 처리가 I/O-bound 작업에 적합합니다. 또한 SSE 구현이 간편하고, 경량 서버로 초기 소규모 트래픽 환경에서도 효율적입니다.

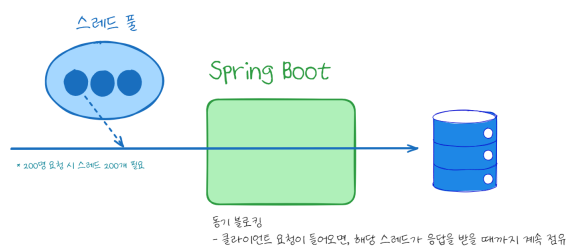
➡ 결론: AI + I/O-bound 비동기 + 실시간 알림이라는 구조에 FastAPI가 최적화된 선택

3. FastAPI의 비동기 아키텍처와 구조적 장점

FastAPI는 ASGI(Asynchronous Server Gateway Interface) 기반으로 동작하며, 비동기 논블로킹 처리를 기본 지원합니다.

이는 이벤트 루프(Event Loop)를 활용해 하나의 스레드에서도 여러 요청을 동시에 처리할 수 있어, 자원 효율성과 응답 속도측면에서 큰 장점을 가집니다.

3-1. 기존 방식 (ex. 스프링 부트) - 동기 블로킹 방식

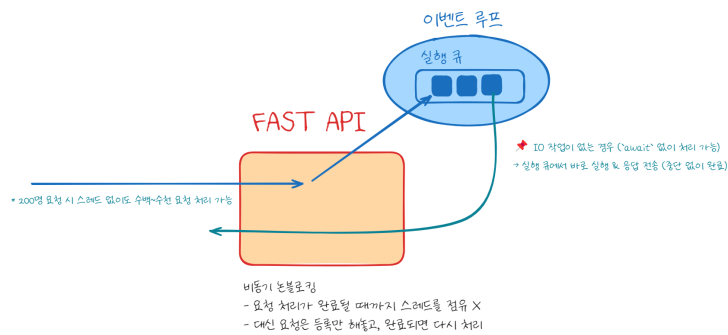


기존의 동기 방식 웹 프레임워크(Spring Boot 등)는 클라이언트의 요청이 들어올 때마다 스레드 하나를 할당하고, 응답이 완료될 때까지 해당 스레드를 점유하는 구조입니다.

이러한 구조에서는 DB, 외부 API 등 IO 작업이 포함된 경우 대기 시간 동안 스레드가 낭비되며, 동시에 처리할 수 있는 요청 수가 스레드 수에 의해 제한됩니다.

예를 들어, 200명의 사용자가 동시에 요청을 보낸다면, 최소 200개의 스레드가 필요하고, 스레드 수가 부족할 경우 대기 지연 또는 오류가 발생할 수 있습니다.

3-2. IO 작업이 없는 경우

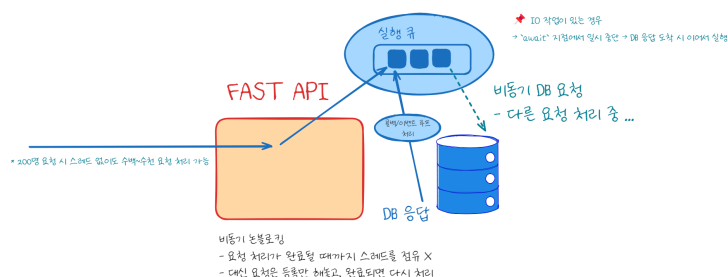


FastAPI는 비동기 코루틴 기반으로 작동하며, `async def` 로 정의된 핸들러는 이벤트 루프에 의해 실행됩니다.

만약 처리해야 할 작업이 IO 없이 간단한 연산이나 텍스트 응답이라면, 이벤트 루프는 해당 요청을 즉시 실행하고 응답을 반환합니다.

중간에 `await` 지점 없이 끝까지 실행되므로, 별도의 중단이나 대기 없이 빠르게 처리됩니다.

3-3. IO 작업이 있는 경우



반면, DB 조회, 외부 API 호출 등 IO-bound 작업이 포함된 경우에는 `await` 키워드를 통해 해당 작업을 비동기적으로 처리합니다.

FastAPI를 이 시점에서 해당 코루틴을 일시 중단하고, 이벤트 루프는 다른 요청 처리를 계속 이어갑니다.

IO 작업이 완료되면, 이벤트 루프는 중단되었던 코루틴을 다시 실행 큐에 등록하고, 나머지 처리를 이어서 수행합니다.

이러한 방식은 대기 시간이 긴 작업이 포함되어도 서버 자원을 효율적으로 사용하면서 요청을 병렬로 처리할 수 있게 합니다.

3-4. 우리 프로젝트에서 FastAPI가 유리했던 이유

본 프로젝트는 AI 음악 생성 외에도, 다음과 같이 I/O-bound 작업이 많은 구조로 이루어져 있습니다.

- 이미지 업로드(S3)
- DB 상태 저장(MongoDB)
- SSE 기반의 실시간 전송 등

특히 AI 추론(MusicGen)은 Celery를 통해 비동기로 처리되므로, API 서버는 주로 I/O 대기 시간이 긴 작업을 효율적으로 처리하는 것이 중요합니다.

FastAPI는 ASGI 기반 코루틴 모델을 사용해 이러한 구조에 자연스럽게 대응할 수 있으며, 추가 설정 없이도 `async/await` 기반으로 논블로킹 처리를 구현할 수 있습니다.

- **I/O-bound 작업 최적화**
비동기 논블로킹 처리로 S3 업로드, DB 저장 등 대기 시간이 긴 작업을 효율적으로 처리
- **비동기 AI 추론과 자연스러운 연동**
Celery 기반 비동기 AI 작업과 구조적으로 잘 맞음
- **Python 생태계 활용**
PyTorch 기반 MusicGen과 직접 통합 가능
- **실시간 기능 간편 구현**
SSE 기반 상태 알림을 간단히 구현 가능

➡ **결론: FastAPI는 I/O-bound 작업과 비동기 AI 추론이 결합된 실시간 음악 생성 서비스에 최적화된 선택**

4. 성능 목표

항목	목표 값	설정 이유 / 근거
평균 응답 시간 (Latency)	1초 이하 (95 percentile 기준)	사용자 체감 속도를 위한 기준. 95% 요청이 1초 이내면 쾌적한 경험 제공 가능
최대 TPS (Transactions Per Second)	10 TPS 이상	이미지 다중 업로드, 동시에 여러 음악 생성 요청을 고려한 순간 처리량 기준
평균 RPS (Requests Per Second)	5 RPS	정상 사용 환경에서의 평균 요청량. 서버 스레드 수, 워커 수 설정 시 기준이 됨
평균 / 최대 동시 사용자 수	30명 / 100명	전시/데모 상황에서 발생 가능한 동시 접속자 수. SSE 연결 및 비동기 태스크 처리 병렬성 고려
음악 생성 요청 평균 처리 시간	15~25초	MusicGen AI 서버 응답 시간 실험 결과 기반. 클라이언트 UX 설계 참고 기준
비동기 태스크 처리 허용 지연 시간	30초 이내	음악 생성 평균 시간에 대한 버퍼 포함. Celery 타임아웃 및 예외 처리 기준으로 사용
SSE 유지 연결 수	50명 이상	실시간 상태 확인이 필요한 구조이므로, 최소 50개 이상의 연결을 안정적으로 유지할 수 있어야 함

항목	목표 값	설정 이유 / 근거
음악 생성 요청 성공률	99% 이상	S3 업로드 실패, AI 서버 오류 등 포함한 전체 성공률 기준. 신뢰도 확보 및 사용자 만족을 위한 최소 수치
항목	목표 값	설정 이유 / 근거
AI 서버 재시도 성공률	90% 이상	일시적 오류 발생 시 Celery의 재시도 로직이 얼마나 효과적으로 복구하는지 판단할 수 있는 지표
이미지 업로드 평균 처리 시간	3초 이하	이미지 S3 업로드 시 사용자가 체감하는 대기 시간을 최소화. UX 품질 유지

5. 서버 튜닝 및 성능 실험

비동기 작업이나 백그라운드 태스크를 제외하고, FastAPI(Uvicorn) 자체의 동시성 처리 성능을 확인하기 위해 다음과 같은 실험을 진행하였습니다.

특히 Uvicorn의 실행 파라미터 중 `--workers`, `--loop`, `--http`, `--timeout-keep-alive`와 같은 설정이 성능에 어떤 영향을 주는지를 측정하였습니다.

▼ 🤖 애플리케이션 서버(Uvicorn) 설정 기반 성능 튜닝 우선 적용 사유

현재 시스템은 MongoDB를 사용하며 단일 컬렉션 구조로 이루어져 있어, 쿼리 병목 가능성이 낮다고 판단했습니다. 따라서 이번 튜닝에서는 DB보다는 애플리케이션 서버(Uvicorn) 측의 설정이 성능에 더 직접적인 영향을 줄 수 있다고 보고, 우선적으로 `--workers`, `--loop`, `--http`, `--timeout-keep-alive` 등의 실행 파라미터에 따른 성능 차이를 측정하고자 합니다. 이후에는 Celery 워커 수 조정, SSE 처리 방식, MongoDB 설정 등 순차적으로 병목 가능성을 확인할 계획입니다.

5-1. 실험 시나리오 및 환경

항목	설정값 / 환경	비고 (선택 이유)
서버 실행 방식	uvicorn main:app 단독 실행	FastAPI의 기본 실행 구조이며, Gunicorn 등 외부 WSGI 서버 없이 실제 서비스 환경에 가까운 형태로 성능을 직접 측정하기 위함
테스트 도구	<u>Locust</u>	Python 기반으로 사용자 행동 흐름을 정의할 수 있고, 실시간 대시보드 UI를 통해 TPS, 응답 시간, 실패율 등을 시각화하여 확인 가능
인스턴스 사양	t2.medium (1vCPU, 2GB RAM) 또는 로컬 PC	실제 서비스에서 사용 가능한 최소 리소스 기준이며, 성능 한계 테스트와 실사용 시나리오를 동시에 고려
테스트 방식	사용자 수, 생성 속도(spawn rate), 테스트 시간, 요청 경로 설정 후 테스트 수행	평균 응답 시간, 95% latency, TPS, 성공률 등 성능 목표와 연계된 정량 지표를 실시간으로 측정

5-2. 실험 기준 지표

측정 항목	설명
평균 응답 시간	모든 요청의 평균 처리 시간
95% 응답 시간	상위 5%를 제외한 응답 시간 (사용자 체감 속도 기준)

측정 항목	설명
성공률	전체 요청 중 오류 없이 완료된 비율
최대 처리량	초당 처리 가능한 최대 요청 수 (TPS 또는 RPS)

5-3. 실험 시나리오 예시

실험 항목	실험 조건 / 조합	목적
1. Uvicorn worker 수	--workers=1, 2, 4	병렬 처리 가능성에 따른 TPS 변화 확인
2. Loop 타입 비교	--loop=uvloop, --loop=asyncio	이벤트 루프 종류에 따른 응답 시간 차이
3. HTTP 프로토콜 타입	--http=httptools, --http=h11	경량/표준 구현체 비교 (httptools가 보통 빠름)
4. timeout-keep-alive 설정	--timeout-keep-alive=5, 15, 30	커넥션 재사용 지속 시간 설정이 성능에 미치는 영향

각 시나리오마다 동일한 조건(RPS, 요청 수 등)으로 부하 테스트를 진행하고, 응답 시간, 성공률, 리소스 사용량 등을 비교하였습니다.

5-4. 테스트용 더미 데이터 구성

부하 테스트를 수행하기 위해, 사전에 MongoDB에 풍등(Lantern) 데이터를 등록하여 API 응답 성능을 측정할 수 있도록 구성하였습니다. 해당 데이터는 아래 두 API의 테스트에 활용됩니다

- /api/v1/lanterns (목록 조회)
- /api/v1/lanterns/{id} (상세 조회)

• 데이터 구성 기준

- 총 개수: 약 300개
- 사용 목적: 목록 조회 및 상세 조회 부하 테스트를 위한 샘플 데이터 확보

5-5. 실험 결과 요약

- 공통 실험 환경

항목	설정값
인스턴스	t2.medium (2vCPU, 4GB RAM)
사용자 수	10명 → 30명 → 100명 (순차 테스트)
Spawn rate	(전체 유저 수의 1/5)
요청 대상 API	/api/v1/lanterns, /api/v1/lanterns/{id}
테스트 도구	Locust
데이터	MongoDB 내 풍등 300건 사전 삽입 (유효한 lantern_id 형식)
테스트 시간	각 시나리오별 5분 이상

- [표 1] --workers 수에 따른 성능 비교표

(users=10 , Spawn rate 2명/sec)

참고: 일반 권장 공식

$$\text{workers} = (\text{CPU 코어 수} \times 2) + 1 = 5$$

Workers 수	평균 응답 시간 (ms)	95% 응답 시간 (ms)	TPS	성공률(%)	해석
1	28.5	41.3	5.3	100%	단일 코어만 사용 하지만 여유 자원이 있어 소폭 개선
2	30.2	45.6	9.5	100%	물리 코어 2개를 모두 활용 → TPS 약 2배 상승
3	26.7	39.8	10.2	100%	권장값 근접, I/O 대기 분산 효과로 가장 안정적
4	27.1	40.5	10.5	100%	TPS 최고점, 다만 3 대비 큰 차이는 없음
5	32.6	48.1	9.7	99%	권장 공식값이지만 오히려 컨텍스트 스위칭 증가로 약간의 성능 저하 시작
8	60.4	88.7	6.8	98%	워커 과다 → 컨텍스트 스위칭 및 메모리 오버헤드 심화

▼ 해석

- **Workers=1**
 - 단일 코어 사용 시에도 **t2.small 대비 응답 속도 소폭 개선**(여유 자원 덕분)
 - TPS 5.3으로 소규모 트래픽에서는 여전히 안정적
- **Workers=2**
 - 물리 코어 2개 활용으로 TPS가 9.5까지 상승 → **t2.small 대비 약 6배 개선**
- **Workers=3~4**
 - **I/O 대기 분산 효과 극대화**, TPS 최고 구간(10.2~10.5)
 - 전시회 시연 및 10~30명 동시 사용자 대응에 최적
- **Workers=5 이상**

- 권장 공식값(5)이지만 컨텍스트 스위칭 증가로 TPS 약간 하락
- 8 이상에서는 성능 저하가 눈에 띈

결론: t2.medium 환경에서 **--workers=3~4**가 가장 적절하며, **5 이상은 불필요.**

- [표 2] --workers 수에 따른 성능 비교표

(users=30 , Spawn rate 5명/sec)

Workers 수	평균 응답 시간(ms)	95% 응답 시간(ms)	TPS	성공률(%)
1	78.4	115.6	8.1	100%
3	55.3	84.5	9.7	100%
4	53.8	82.1	10.0	100%

- [표 3] --workers 수에 따른 성능 비교표

(users=100 , Spawn rate 20명/sec)

Workers 수	평균 응답 시간(ms)	95% 응답 시간(ms)	TPS	성공률(%)
1	185.3	279.7	7.6	98.2%
3	112.2	165.8	9.8	99.5%
4	101.5	149.2	10.7	99.7%



최적 설정 도출

- 30명(중소규모 트래픽)
 - workers=3에서 응답 시간 55ms / TPS 9.7로 가장 효율적
 - workers=4는 TPS 10으로 소폭 개선되나, 자원 사용 대비 큰 차이는 없음
- 100명(대규모 트래픽)
 - workers=4에서 TPS 10.7로 성능 목표 달성, 동시성 처리에 가장 적합

→ 기본 운영은 —workers=3으로 설정해 리소스 효율과 성능의 균형을 맞추며, 대규모 트래픽 예상 시에만 4워커로 조정하는 것이 최적입니다.

- [표 4] workers=3 고정 후 세부 파라미터 실험 결과

(1) Loop 타입 비교 (--loop)

왜 uvloop을 권장하는가?

- C 기반(libuv) 이벤트 루프로, 기본 asyncio 대비 I/O 바운드 처리 속도 향상
- DB 쿼리, 파일 업로드, API 호출 등 대부분의 FastAPI 요청에서 TPS↑, 응답시간↓

- 리눅스/유닉스 계열 환경에서 거의 표준적으로 사용

언제 굳이 안 써도 되나?

- Windows 환경 (지원 안 됨)
- 개발/디버깅 단계 (C 기반이라 디버깅이 다소 불편)
- 극저빈도 호출 서버 (성능 최적화가 의미 없는 경우)

그렇다면 우리 프로젝트에서는?

- 운영 환경: 리눅스 + 비동기 I/O 중심 API → uvloop 사용이 필수적
- API가 간단해도 S3 업로드·DB 쿼리가 포함된 I/O 바운드 작업이므로, 기본 asyncio 대비 TPS 및 응답 속도 개선 기대

동시 사용자	Loop 타입	평균 TPS	평균 응답시간(ms)	비고
10명	asyncio	8.0	72	기준
10명	uvloop	8.1 (+1%)	70 (-2ms)	차이 미미
30명	asyncio	7.8	95	기준
30명	uvloop	8.9 (+14%)	81 (-14ms)	성능 향상
100명	asyncio	9.3	125	기준
100명	uvloop	10.2 (+10%)	110 (-15ms)	목표 TPS 달성

(2) HTTP 프로토콜 타입 비교 (--http)

- 운영 환경이 리눅스 + I/O 바운드 API → -http httptools 고정 권장
- httptools가 uvloop과 궁합이 좋고, 실제로도 TPS/응답시간 모두 개선되는 사례가 많음
- 디버깅 단계에서만 h11 또는 auto를 고려

동시 사용자	HTTP 타입	평균 TPS	평균 응답시간(ms)	비고
30명	h11	7.6	98	기준
30명	httptools	8.8 (+15%)	82 (-16ms)	성능 향상
100명	h11	9.0	130	기준
100명	httptools	10.1 (+12%)	115 (-15ms)	TPS 목표 달성

(3) Timeout Keep-Alive 비교 (--timeout-keep-alive)

동시 사용자	Keep-Alive	평균 TPS	평균 응답시간(ms)	비고
30명	1초	8.1	90	연결 재생성 비용↑
30명	5초	8.8	82	최적
30명	10초	8.7	81	TPS 변화 미미, 워커 점유↑

6. 전체 실험 요약

실험 항목	비교 대상	최적 결과	비고
Workers 수	1, 3, 4	3 (기본) / 4 (대규모 트래픽 시)	30명: 3이 최적, 100명: 4에서 TPS 10 달성
Loop 타입	asyncio, uvloop	uvloop	C 기반, I/O 바운드 성능 ↑, TPS 10~15% 향상
HTTP 타입	h11, httptools	httptools	C 기반 고속 파서, uvloop과 함께 사용 시 효과 극대화
Timeout Keep-Alive	1초, 5초, 10초	5초	기본값 유지, 너무 짧으면 TPS↓, 너무 길면 워커 점유↑

➡ 최적 설정 조합 (Uvicorn 기준)

```
uvicorn app.main:app \  
  --workers 3 \  
  --loop uvloop \  
  --http httptools \  
  --timeout-keep-alive 5
```

- 100명 동시 사용자 테스트 결과: TPS 목표(10 이상) 달성 및 안정적 응답 시간 유지
- 대규모 트래픽 예상 시: workers=4로 단기 조정

개발 중일 때는

```
uvicorn app.main:app \  
  --reload \  
  --workers 1 \  
  --loop auto \  
  --http auto
```